GAME DESIGN DOCUMENT

Dune: Arrakis Dominion Distributed

Sistema de Recursos y Producción

Versión: 0.1
Fecha: Abril 2026
Motor: Godot 4.x
Plataforma: PC/Web

ÍNDICE

1. Visión General
2. Sistema de Recursos
3. Cadena de Producción
4. Mecánicas de Extracción
5. Sistema de Rondas
6. Interfaz de Usuario
7. Implementación en Godot

1. VISIÓN GENERAL

1.1 Premisa del Juego

El jugador controla una **Casa Menor** asignada a un sector de Arrakis. Debe gestionar la extracción de recursos, construir enclaves, manejar criaturas y recibir visitantes mientras cumple cuotas de producción y evita conflictos diplomáticos.

1.2 Objetivos del Sistema de Recursos

- Crear un **circuito económico** creíble dentro del universo Dune
- Ofrecer **decisiones significativas** sobre asignación de recursos
- Mantener **accesibilidad** (1-2 horas por partida)
- Justificar narrativamente cada mecánica

1.3 Tipos de Recursos

Recurso	Descripción	Rareza	Uso Principal
Especia (Melange)	Recurso primario, adictivo, extensible de vida	Muy rara	Moneda, exportación, comercio
Solaris	Energía solar captada	Abundante	Combustible de instalaciones
Agua	Recurso crítico en Arrakis	Escasa	Supervivencia, producción
Materiales	Construcción básica	Común	Edificios, mantenimiento
Conocimiento	Datos sobre fauna,地形, amenazas	Variable	Investigación, eventos

2. SISTEMA DE RECURSOS

2.1 Recurso Primario: Especia (Melange)

Propiedades

- **Unidad:** Kg de Especia
- **Producción:** Variable según sector y condiciones
- **Decaimiento:** La especia cosechada pierde 5% de valor por ronda si no se procesa/vende
- **Almacenamiento:** Requiere contenedores especiales (costosos)

Modelo Matemático de Producción

```
Producción_Especia =
    (Afloramientos_Sector × Factor_Clima × Factor_Gusanos)
    - Costo_Operacional
    × Eficiencia_Enclaves
```

Variables:

- `Afloramientos_Sector` : 1-5 por sector (fijos al inicio)
- `Factor_Clima` : 0.3 - 1.5 (basado en eventos de tormenta)
- `Factor_Gusanos` : 0.5 - 2.0 (si hay gusanos activos, mayor producción)
- `Eficiencia_Enclaves` : 0.5 - 2.0 (basado en nivel de instalaciones)
- `Costo_Operacional` : 10-50 Solaris por ronda

Justificación Narrativa

> *"La especia no se 'mina'. Se observa, se espera, se arriesga. El desierto decide cuándo y dónde." — Fragmento de "Manual del Operador de Casa Menor"*

En Dune canon, la especia:

- Se forma en las profundidades del desierto

- Los gusanos de arena son esenciales para su ciclo vital
- Las "trampas de arena" indican afloramientos
- Las tormentas pueden destruir o revelar depósitos

2.2 Recurso Secundario: Solaris

Propiedades

- **Unidad:** Unidades de Energía (UE)
- **Producción:** Constante si hay paneles solares activos
- **Almacenamiento:** Baterías con capacidad limitada
- **Decaimiento:** No se deteriora

Modelo de Producción

```
Producción_Solaris =
    Paneles_Activos × 10 UE/panel/ronda
    - Consumo_Instalaciones
    = Excedente
```

Justificación Narrativa

Arrakis recibe el doble de radiación solar que la Tierra debido a sus dos soles. Los Fremen desarrollaron windtraps y captadores solares. Las tormentas de arena reducen la eficiencia.

2.3 Recurso Crítico: Agua

Propiedades

- **Unidad:** Litros (l)
- **Producción:** Limitada (windtraps, reciclaje)
- **Consumo:** Personal, instalaciones, criaturas
- **Almacenamiento:** Tanques limitados

Modelo de Consumo

```
Consumo_Agua =
    Personal × 2 l/persona/ronda
    + Criaturas × 5-20 l/criatura/ronda
```

```
    + Instalaciones × 10-100 l/ronda
    = Total_Consumo
```

Justificación Narrativa

> *"El agua es vida en Arrakis. Cada gota cuenta." — Proverbio Fremen*

En el universo Dune, el agua es el recurso más valioso en el desierto. Los Fremen reciclan todo el líquido corporal.

2.4 Recurso de Construcción: Materiales

Propiedades

- **Unidad:** Toneladas (T)
- **Producción:** Extracción de rocas, reciclaje de escombros
- **Uso:** Construcción y mejoras

Modelo

```
Materiales_Nuevos =
    Minas_Activas × 5 T/ronda
    + Reciclaje × 0.3 × Materiales_Demolidos
```

---

3. CADENA DE PRODUCCIÓN

3.1 Flujo General

```
[RECOURSOS BRUTOS] → [PROCESAMIENTO] → [RECURSOS PROCESADOS] → [USO]
        ↓                    ↓                    ↓
  Windtraps          Refinerías          Almacenes
  Minas              Destilerías         Exportación
  Paneles solares    Laboratorios        Consumo interno
  Trampas de arena   Talleres
```

3.2 Estaciones de Producción

Fase 1: Extracción

Instalación	Recurso Bruto	Requisitos	Output/ronda
Trampa de Especia	Afloramientos	Permisos, trabajadores	5-20 kg especia/ronda
Panel Solar	Luz solar	Espacio despejado	10 UE/ronda
Windtrap	Aire húmedo	Ubicación en quebradas	50 l agua/ronda
Pozo de Materiales	Rocas	Herramientas	5 T materiales/ronda

Fase 2: Procesamiento

Instalación	Input	Output	Costo
Refinería de Especia	Especia bruta	Especia procesada (+20% valor)	30 UE/ronda
Destilería de Agua	Agua impura	Agua purificada (+10% eficiencia)	15 UE/ronda
Taller	Materiales	Herramientas/Componentes	20 UE/ronda

Fase 3: Almacenamiento

- **Silo de Especia**: Capacidad 100 kg
- **Baterías Solares**: Capacidad 500 UE
- **Tanque de Agua**: Capacidad 1,000 l
- **Almacén de Materiales**: Capacidad 200 T

3.3 Balanceo del Sistema

Para mantener accesibilidad, el juego tendrá:

1. **Límites claros** de producción por instalación
2. **Sin microgestión** - las instalaciones funcionan automáticamente
3. **Decisiones estratégicas** sobre qué construir y cuándo
4. **Eventos que interrumpen** el flujo (tormentas, migraciones de gusanos)

4. MECÁNICAS DE EXTRACCIÓN

4.1 Exploración del Sector

Al inicio de cada partida, el jugador recibe un **sector procedural** que contiene:

- **Afloramientos de Especia** (marcados como puntos de interés)
- **Quebradas y Zonas Protegidas** (para windtraps)
- **Zonas de Alta Radiación** (para paneles solares)
- **Yacimientos de Rocas** (para minas)

Mecánica de Exploración

```
Acciones_de_Exploración = 3/ronda
Tiempo_de_Exploración = 1 acción = 1 zona revelada
```

Revelar una zona muestra:

- Tipo de recurso
- Cantidad estimada
- Peligros asociados

4.2 Colocación de Instalaciones

El jugador selecciona ubicaciones en un **mapa isométrico** del sector:

1. **Zonas abiertas**: Solo paneles solares
2. **Quebradas**: Windtraps, enclaves
3. **Afloramientos**: Trampas de especia
4. **Acantilados**: Torres de vigilancia

Restricciones de Ubicación

- Cada instalación requiere **espacio específico**
- Zonas peligrosas requieren **personal de riesgo**
- Algunas zonas solo se desbloquean con **tecnología**

4.3 Ciclo de Producción Automatizado

Una vez colocadas, las instalaciones funcionan automáticamente:

```
[Ronda N]
  ├─> Mantenimiento automático (costo de materiales)
  ├─> Producción según eficiencia
  ├─> Eventos aleatorios (20% probabilidad)
  │      ├─ Tormenta (reduce producción 50%)
  │      ├─ Gusano cerca (aumenta especia, destruye paneles)
  │      └─ Hallazgo (recurso extra)
  └─> Resultado aplicado al inventario
```

5. SISTEMA DE RONDAS

5.1 Estructura de una Ronda

```
┌─────────────────────────────────────────────────────────────┐
│                        RONDA [N]                            │
└─────────────────────────────────────────────────────────────┘
│                                                             │
│  [FASE 1: EVENTOS]                                          │
│    └─> Eventos climáticos                                   │
│    └─> Migraciones de fauna                                 │
│    └─> Visitantes llegan                                    │
│                                                             │
│  [FASE 2: PRODUCCIÓN]                                       │
│    └─> Cálculo de recursos                                  │
│    └─> Aplicar eficiencia                                   │
│    └─> Eventos afectan producción                           │
│                                                             │
│  [FASE 3: DECISIONES]                                       │
│    └─> Construir/Mantener instalaciones                     │
│    └─> Asignar personal                                     │
│    └─> Negociar con visitantes                              │
│                                                             │
│  [FASE 4: RESULTADOS]                                       │
│    └─> Resumen de producción                                │
│    └─> Estado de recursos                                   │
│    └─> Próxima ronda                                        │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

5.2 Duración de Ronda

- **1 Ronda = 1 ciclo de producción**
- **Partida completa = 12-20 rondas** (1-2 horas)
- **Condiciones de fin**: Cuota cumplida, bankruptcy, o decisión del jugador

5.3 Eventos Aleatorios

Evento	Probabilidad	Efecto	Duración
Tormenta de Arena	15%	-50% producción global	1 ronda
Gusano Errante	10%	Destruye 1 instalación, +especia en zona	Instantáneo
Afloramiento Oculto	20%	Revela nuevo punto de especia	Permanente
Cometa de Especia	5%	+100% producción esta ronda	1 ronda
Fallo Mecánico	10%	-30% eficiencia instalación	2 rondas
Visitante VIP	15%	Oportunidad comercial/diplomática	Esta ronda

6. INTERFAZ DE USUARIO

6.1 Pantalla Principal (HUD)

```
[ESCUDO CASA]              RONDA 5/15              [OPCIONES]

RECURSOS                              MINI-MAPA

Especia:  67/100 kg                 [Mapa isométrico]
Solaris:  450/500
Agua:     780/1k                    ★ Sector del jugador
Materiales:  45/200 T

[MAPA COMPLETO]           [CONSTRUIR]           [PERSONAL]

LOG DE EVENTOS
> Tormenta de arena pasó por el Sector Este
> Llega diplomático de Casa Vernius
> Producción de especia: +15kg
```

6.2 Pantalla de Construcción

```
CONSTRUIR INSTALACIÓN                                  [X]

CATEGORÍAS:
[Producción] [Almacenamiento] [Investigación] [Defensa]

   PANEL SOLAR        WINTRAP          TRAMPA ARENA
   🌞 10 UE/ronda     💧 50l/ronda     ⊕ 15kg/ronda

   Costo: 20T         Costo: 30T       Costo: 50T
   维护: 2 UE         维护: 5 UE       维护: 10 UE
```

```
[CONSTRUIR]   [CONSTRUIR]   [CONSTRUIR]

[CANCELAR]                  [SIGUIENTE]
```

6.3 Flujo de Ronda Visual

```
                RONDA 5
                   │
        ┌──────────┴──────────┐
        ▼          ▼          ▼
   EVENTOS    PRODUCCIÓN  DECISIONES
[Tormenta]   +15 kg ⊕   [Construir?]
[Gusano]     +100 UE 🌞  [Asignar?]
[VIP]        +50 l 💧    [Negociar?]
        └──────┬──────┘
               ▼
              RESUMEN
           Recursos ▲
           Nueva ronda
```

7. IMPLEMENTACIÓN EN GODOT

7.1 Estructura del Proyecto

```
res://
├── scenes/
│   ├── main/
│   │   ├── Main.tscn           # Escena principal
│   │   └── HUD.tscn            # Interfaz superior
│   ├── map/
│   │   ├── SectorMap.tscn      # Mapa isométrico
│   │   └── Tile.tscn           # Casilla individual
│   ├── ui/
│   │   ├── ResourcePanel.tscn  # Panel de recursos
│   │   ├── BuildMenu.tscn      # Menú de construcción
│   │   └── EventLog.tscn       # Registro de eventos
│   └── entities/
│       ├── Installation.tscn   # Clase base instalaciones
│       └── types/              # Tipos específicos
├── scripts/
│   ├── core/
│   │   ├── GameManager.gd      # Controlador principal
│   │   ├── RoundManager.gd     # Lógica de rondas
│   │   └── ResourceManager.gd  # Sistema de recursos
│   ├── systems/
│   │   ├── ProductionSystem.gd # Cadena de producción
│   │   ├── EventSystem.gd      # Sistema de eventos
│   │   └── BuildSystem.gd      # Construcción
│   └── ui/
│       ├── HUDSystem.gd        # Actualización del HUD
│       └── MapInteractions.gd  # Click en mapa
├── resources/
│   ├── installations/          # Datos de instalaciones
│   ├── events/                 # Definiciones de eventos
│   └── sectors/                # Configuración de sectores
└── assets/
    ├── sprites/
    ├── tilesets/
    └── fonts/
```

7.2 Clase ResourceManager (Core)

```
class_name ResourceManager
extends Node

signal resource_changed(resource_type: String, amount: float)
signal capacity_warning(resource_type: String)

enum ResourceType {ESPECIA, SOLARIS, AGUA, MATERIALES, CONOCIMIENTO}

@export var resources: Dictionary = {
    ResourceType.ESPECIA: 0.0,
    ResourceType.SOLARIS: 100.0,
    ResourceType.AGUA: 200.0,
    ResourceType.MATERIALES: 50.0,
    ResourceType.CONOCIMIENTO: 0.0
}

@export var capacities: Dictionary = {
    ResourceType.ESPECIA: 100.0,
    ResourceType.SOLARIS: 500.0,
    ResourceType.AGUA: 1000.0,
    ResourceType.MATERIALES: 200.0,
    ResourceType.CONOCIMIENTO: 50.0
}

func modify_resource(type: ResourceType, amount: float) -> bool:
    var new_value = resources[type] + amount
    var capacity = capacities[type]

    if amount > 0 and new_value > capacity:
        resource_changed.emit(type, capacity - resources[type])
        resources[type] = capacity
        capacity_warning.emit(type)
        return false

    resources[type] = clamp(new_value, 0, capacity)
    resource_changed.emit(type, amount)
    return true

func get_resource(type: ResourceType) -> float:
    return resources[type]

func get_capacity(type: ResourceType) -> float:
    return capacities[type]

func get_fill_percentage(type: ResourceType) -> float:
    return resources[type] / capacities[type]
```

7.3 Clase Installation (Base)

```
class_name Installation
extends Node2D

signal production_complete(output: Dictionary)
signal efficiency_changed(new_efficiency: float)

@export var installation_type: String
@export var efficiency: float = 1.0
@export var is_active: bool = true

var base_production: Dictionary = {}
var maintenance_cost: Dictionary = {}
var current_round_production: float = 0.0

func _ready():
    setup_installation()

func setup_installation() -> void:
    # Cargar datos del recurso de instalación
    var data = load_installation_data(installation_type)
    base_production = data["production"]
    maintenance_cost = data["maintenance"]

func process_production() -> Dictionary:
    if not is_active:
        return {}

    var output = {}
    for resource_type in base_production.keys():
        var amount = base_production[resource_type] * efficiency
        output[resource_type] = amount

    current_round_production = output.values().reduce(func(a, b): return a + b, 0.0)
    production_complete.emit(output)
    return output

func apply_event(event_type: String, intensity: float) -> void:
    match event_type:
        "storm":
            efficiency = clamp(efficiency * (1.0 - intensity), 0.1, 1.0)
        "sandworm":
            efficiency = clamp(efficiency * (1.0 + intensity), 0.5, 2.0)
        "malfunction":
            efficiency = clamp(efficiency - intensity, 0.1, 1.0)

    efficiency_changed.emit(efficiency)

func get_maintenance_cost() -> Dictionary:
    return maintenance_cost.duplicate()
```

```
func upgrade() -> void:
    efficiency = clamp(efficiency + 0.1, 1.0, 2.0)

func destroy() -> void:
    queue_free()
```

7.4 Sistema de Producción

```
class_name ProductionSystem
extends Node

@export var resource_manager: ResourceManager
@export var installations: Array[Installation] = []

func process_round() -> Dictionary:
    var round_results = {
        "production": {},
        "consumption": {},
        "events": []
    }

    # Calcular producción total
    for installation in installations:
        if installation.is_active:
            var production = installation.process_production()
            for resource_type in production.keys():
                round_results["production"][resource_type] = \
                    round_results["production"].get(resource_type, 0.0) +
production[resource_type]

    # Aplicar costos de mantenimiento
    var maintenance = calculate_maintenance()
    round_results["consumption"] = maintenance

    # Aplicar cambios a recursos
    for resource_type in round_results["production"].keys():
        resource_manager.modify_resource(resource_type, round_results["production"]
[resource_type])

    for resource_type in maintenance.keys():
        resource_manager.modify_resource(resource_type, -maintenance[resource_type])

    return round_results

func calculate_maintenance() -> Dictionary:
    var total_maintenance = {}

    for installation in installations:
```

```
        if installation.is_active:
            var costs = installation.get_maintenance_cost()
            for resource_type in costs.keys():
                total_maintenance[resource_type] = \
                    total_maintenance.get(resource_type, 0.0) + costs[resource_type]

    return total_maintenance
```

7.5 EventSystem

```
class_name EventSystem
extends Node

signal event_triggered(event_data: Dictionary)

@export var installations: Array[Installation] = []
@export var sector_map: Node2D

var event_definitions: Array[Dictionary] = [
    {
        "id": "storm",
        "name": "Tormenta de Arena",
        "probability": 0.15,
        "duration": 1,
        "effects": {"efficiency": -0.5, "production": -0.5}
    },
    {
        "id": "sandworm",
        "name": "Gusano Errante",
        "probability": 0.10,
        "duration": 0,
        "effects": {"destroy_chance": 0.1, "especias_bonus": 1.5}
    },
    {
        "id": "spice_bloom",
        "name": "Floración de Especia",
        "probability": 0.20,
        "duration": 1,
        "effects": {"especias": 1.5}
    },
    {
        "id": "malfunction",
        "name": "Fallo Mecánico",
        "probability": 0.10,
        "duration": 2,
        "effects": {"efficiency": -0.3}
    }
```

```
]

func roll_for_events() -> Array[Dictionary]:
    var triggered_events = []

    for event in event_definitions:
        if randf() < event["probability"]:
            triggered_events.append(event)
            event_triggered.emit(event)

    return triggered_events

func apply_event_to_system(event: Dictionary) -> void:
    match event["id"]:
        "storm":
            for installation in installations:
                installation.apply_event("storm", event["effects"]["efficiency"])

        "sandworm":
            apply_sandworm_event(event)

        "spice_bloom":
            apply_spice_bonus(event["effects"]["especias"])

        "malfunction":
            var target = installations[randi() % installations.size()]
            target.apply_event("malfunction", event["effects"]["efficiency"])

func apply_sandworm_event(event: Dictionary) -> void:
    # Posibilidad de destruir instalación
    if randf() < event["effects"]["destroy_chance"]:
        var target = installations[randi() % installations.size()]
        if target.installation_type != "fortress":
            target.destroy()

    # Bonus de especia si hay trampas
    var bonus = event["effects"]["especias_bonus"]
    for installation in installations:
        if installation.installation_type == "spice_trap":
            installation.apply_event("sandworm", bonus)

func apply_spice_bonus(multiplier: float) -> void:
    # Este bonus se aplica en el próximo process_round
    pass
```

7.6 RoundManager

```
class_name RoundManager
extends Node

signal round_started(round_number: int)
signal round_ended(round_results: Dictionary)
signal game_ended(final_score: Dictionary)

@export var current_round: int = 1
@export var max_rounds: int = 15

@export var resource_manager: ResourceManager
@export var production_system: ProductionSystem
@export var event_system: EventSystem

var is_processing: bool = false

func start_round() -> void:
    if is_processing or current_round > max_rounds:
        return

    is_processing = true
    round_started.emit(current_round)

    # FASE 1: Eventos
    var events = event_system.roll_for_events()
    for event in events:
        event_system.apply_event_to_system(event)

    # FASE 2: Producción
    var results = production_system.process_round()
    results["events"] = events

    # FASE 3: Decisiones del jugador (esperar input)
    # Esto se maneja via UI en el juego real

    round_ended.emit(results)
    current_round += 1
    is_processing = false

func check_end_conditions() -> bool:
    # Verificar si se cumple cuota de especia
    var target_spice = 200.0 * (current_round / max_rounds)
    if resource_manager.get_resource(ResourceManager.ResourceType.ESPECIA) >= target_spice:
        return true

    # Verificar bankruptcy
    if resource_manager.get_resource(ResourceManager.ResourceType.SOLARIS) < 0:
        return true

    # Verificar game over por agua
```

```
    if resource_manager.get_resource(ResourceManager.ResourceType.AGUA) <= 0:
        return true

    return false

func end_game() -> Dictionary:
    var score = {
        "total_rounds": current_round - 1,
        "final_resources": resource_manager.resources.duplicate(),
        "quota_fulfilled": resource_manager.get_resource(ResourceManager.ResourceType.ESPECIA)
>= 200.0,
        "installations_built": count_installations(),
        "events_survived": count_events()
    }

    game_ended.emit(score)
    return score
```

ANEXO A: Tablas de Balanceo

Producción Base por Instalación

Instalación	Recurso	Cantidad Base	Mantenimiento (UE)
Panel Solar	Solaris	10/ronda	2
Windtrap	Agua	50/ronda	5
Trampa de Especia	Especia	15/ronda	10
Pozo de Materiales	Materiales	5/ronda	3
Refinería	Procesa especia	+20% valor	30
Almacén	Capacidad	+50%	5

Costos de Construcción

Instalación	Materiales	Agua	Tiempo (rondas)
Panel Solar	20T	10l	1
Windtrap	30T	50l	2
Trampa de Especia	50T	100l	3
Refinería	80T	200l	4

Probabilidades de Eventos

Evento	Probabilidad Base	Modificador Sector
Tormenta	15%	+5% zonas abiertas
Gusano Errante	10%	+10% cerca de afloramientos
Afloramiento Oculto	20%	Fijo
Cometa de Especia	5%	-5% si ya hubo este turno
Fallo Mecánico	10%	+5% instalaciones viejas

PRÓXIMOS PASOS

1. **Revisar y validar** este documento de diseño
2. **Implementar prototipo** en Godot 4.x con las clases básicas
3. **Playtesting** del loop de producción
4. **Iterar** basado en feedback

Documento generado para: Dune: Arrakis Dominion Distributed Basado en el universo creado por Frank Herbert